机器人控制系统 APP 使用手册

Balance Beam Tech · 实时人形/双臂机器人控制平台

适用机型：humanoid（人形） / dualarm（双臂）

版本：2026-10-14

读者：现场操作员 + 集成/部署工程师

1. 系统简介

1.1 架构速览

系统由 **Launcher**（进程启动器）拉起四个实时任务进程，经共享内存（shm）互联， 浏览器 HMI 提供操作界面：

组件	职责
StateMaster（主任务）	主状态机编排、程序供给器（水位喂环）、服务端循环、HMI 服务（:8080）、 系统状态持久化（恢复文件）
Planning	轨迹规划（ppwodyn 管线）、程序/手动双管线、启动门控、断点书签
Servo	轨迹插补 → 微步流（水位节流）、沿路径制动
Fieldbus	EtherCAT 通讯（4ms 周期）、反馈采集、规划停保底

1.2 双机型（flavor）差异

	humanoid	dualarm
运动学	双臂 7 关节 + 手指/拇指 + 腰	双臂 6 关节（CNieTarget）
URDF/配置	humanoid.urdf / humanoid_robot_conf.cfg	2arm_v1.urdf / dualarm_robot_conf.cfg / servo_conf_dualarm.cfg
部署目录	task/app/humanoid/	task/app/dualarm/
实轴情况	现场仅少量实轴，其余为虚拟轴（位置靠记忆保持）	双臂全实轴（绝对编码器）

两个机型的操作界面与流程完全一致；本文差异处单独注明。

2. 启动与关停

2.1 启动

```
cd /home/user/claude/task/app/<flavor>     # humanoid 或 dualarm
sudo ./taskframe_launcher                   # 默认读 cfg/taskframe_launcher.yaml
```

- Launcher 依次拉起 StateMaster →（注册+配置就绪后）→ Planning / Servo / Fieldbus；
- Fieldbus 初始化 EtherCAT（需总线在线；INIT 窗口约 19s）；
- HMI 就绪后浏览器打开 `http://<机器人 IP>:8080`。

2.2 关停

Launcher 终端按 **回车** 或 Ctrl-C：先广播下电（graceful motor off，最多等 3s）， 再停各 worker。切勿直接 kill——状态机会残留共享内存。

> **shm 残留清理**：异常掉电/强杀后，下次启动前若报 `shm_open: Permission denied` 或 Launcher 提示残留，执行：

```
sudo rm /dev/shm/ROBOT_GLOBAL_DATA
```

2.3 重启恢复（2026-10-13 起）

系统在运行中持续把现场写入 `task/app/<flavor>/data/recovery_state.json` （程序原文、执行断点、停止位置、运行模式）。**重启后自动恢复**：

- 程序从断点组重新入环（中断组整组重跑）；
- 虚拟轴位置恢复为上次停止位置（humanoid 现场多为虚拟轴，不再归零）；
- 运行模式（手动/自动）恢复；实轴位置以编码器为准并自动核对（偏差 >0.1 rad 报警）。

> 恢复只恢复上下文，**状态机仍需人工走流程**（初始化→…→使能）， 程序须重新按「启动程序」才运行。关机后请勿物理搬动虚拟轴。

3. 状态机与基本操作

3.1 主状态机

```
UNINITIATED —初始化→ UNCONFIGED —配置→ MOTOR_OFF —上电→ READY
READY —使能(CONTROL_ON)→ MOTION —去使能→ READY —下电→ MOTOR_OFF
任意态 —急停→ ESTOP —急停复位→ MOTOR_OFF; ERROR —重置→ MOTOR_OFF
```

3.2 操作顺序（每次开机）

1. **初始化** → **配置** → **上电** →（各任务状态字同步确认）；
2. **使能**（CONTROL_ON）→ MOTION；
3. AUTO 程序：**进 MOTION 不会自动运行**——见 §4.2 启动门控；
4. 手动模式：下发点动动作即规划执行（无启动门控）。

3.3 错误与急停恢复

- **ERROR**：排障后按「重置」回 MOTOR_OFF，重新上电；
- **急停（ESTOP）**：任意状态可进，机器人在动时先规划停再断链；「急停复位」回 MOTOR_OFF；
- 运动中收到 ERROR/ESTOP 时，Servo 沿路径制动、Fieldbus 保底四步停，停稳才断使能。

4. 界面操作（HMI）

浏览器打开 `http://<机器人 IP>:8080`。页签：操作 / 程序 / 示教 / 监控 / 3D 孪生。

4.1 操作页

区域	内容
状态命令	初始化/配置/上电/下电/使能/去使能/重置/急停、 停止程序（PathStop）、回安全位、暂停/继续、**启动程序**
模式切换	手动模式 / 自动模式（Planning 在 READY/MOTION 时拒绝切换， 须先下电）
队列水位	手动/程序/轨迹/微步四环水位 + 程序启动状态 （待启动/已启动）+ 循环状态
消息日志	命令回执（HTTP 状态 + 拒绝原因）

4.2 程序页

- **构建器**：逐行添加动作组（unitType/motionType/目标值/速度）； 关节动作 motionType=关节，直线 motionType=直线；实时 JSON 预览；
- **下发程序**：全量交给 StateMaster 供给器（不再有 32 组上限， 水位分批入环，恒保 2 空位）；重复下发 = 替换（环内旧程序同步清除）；
- **启动程序**（操作页按钮）： AUTO 门控开门——见 §5.1；
- **循环执行**：勾选「循环执行」+ 次数（或勾「无限循环」）→「开始循环」。 先下发一次，**首轮须人工按「启动程序」**，后续轮自动续跑； 再点「停止循环」或 PathStop 随时停；
- **停止程序**：清程序环 + 停循环 + 断点复位（程序不随重启复活）。

4.3 示教页 / 监控页 / 3D 孪生页

- 示教页：点位与程序存档（服务端 `hmi/data/` 持久化）；
- 监控页：逐轴位置/速度曲线（选中轴环形缓存）；

- 3D 孪生页：URDF 模型 + 实时位姿（FK 反馈驱动）。

5. 关键运行语义

5.1 启动门控（2026-10-13 起）

> AUTO 模式进 MOTION **不自动跑**。须按「启动程序」开门。 清零（回待启动）时机：去使能 / 下电 / ERROR / 急停 / PathStop / 切手动 / 重启。READY 态可提前按（预取开门，使能后即跑）。

5.2 暂停 / 继续

暂停 = 沿路径制动到零速、保持使能；继续 = 以反馈为起点重规划剩余几何 （书签机制）。与启动门控正交。

5.3 程序速度

程序每行的 speed 字段（tcp_m_s / joint_rad_s）为**绝对期望速度**， 规划时收至 min(管线默认, 硬件限)：关节默认 1.05 rad/s、笛卡尔按机型硬件限。 填 0 / 缺省 = 默认。

> 注意：恰好填默认值（tcp 0.25 / joint 1.05）会被管线视为 「未设置」（标记值语义）——微调请避开这两个值。

5.4 手动模式限速

手动点动限速：TCP ⩽ 0.25 m/s、关节 ⩽ 0.3 rad/s（独立手动管线，与程序 管线分离）。

5.5 零位移组

起点==终点的组（如循环重放已在目标的组）自动跳过（不报错、不动）， 日志打 `zero-motion group, skipped`。

6. 配置过程（工程师）

6.1 构建与部署（flavor）

```
cd task/<模块>/build
cmake .. -G Ninja -DBBT_ROBOT_FLAVOR=humanoid   # 或 dualarm
cmake --build .                                  # POST_BUILD 自动部署到
task/app/<flavor>/
```

模块：TaskFrame（库）→ StateMaster / Planning / Servo / Fieldbus / Launcher。 **上游库更新后下游必须重链**（静态库陷阱）。

6.2 配置文件

文件	位置	内容
taskframe_launcher.yaml	app/<flavor>/cfg/	worker 清单（名称/优先级/CPU 绑定）
taskframe_paths.cfg	app/<flavor>/cfg/	urdf / robot_cfg / servo_conf 三键路径
stop_conf.json	app/<flavor>/cfg/	stop_mode（on_path/fast/emergency）+ 安全位
recovery_state.json	app/<flavor>/data/	恢复文件（运行中自动写，勿手改；要清零现场可删除）
URDF / robot_conf / servo_conf	cfg/（工作区根）	机型运动学/驱动参数/总线配置——改动需配套重构建

6.3 常用运维

- 清除恢复现场：`rm task/app/<flavor>/data/recovery_state.json` 后重启；
- 清 shm 残留：`sudo rm /dev/shm/ROBOT_GLOBAL_DATA`；
- HMI 换端口：`state_master_<flavor>.elf --hmi-port=<port>`（launcher yaml 内参数）。

7. 故障处理

日志/现象	含义与对策
shm_open: Permission denied	root 残留 shm → `sudo rm /dev/shm/ROBOT_GLOBAL_DATA`
planning failed: ...（soc N）	规划失败进 ERROR——看同行 func 信息定位（IK 不可达/非法 motionType）， 排障后重置
zero-motion group, skipped	零位移组自动跳过（正常，非故障）
waist data not attached, using waist=0	已知无害告警（臂 FK 腰耦合缺省），可忽略
ring full for task N（N≥4）	事件广播写未注册槽位的既有噪音，功能无害
程序不动 / 队列有组不出段	启动门控未开——操作页按「启动程序」；检查队列水位「程序启动」状态
程序队列空但已下发	检查 HMI 消息日志的逐行拒绝原因（坏行单独报）
Unknown state 刷屏	2026-10-13 已修复（状态映射补齐）——确认 elf 为当日新版

8. 附录

8.1 测试入口

```
cd task/<模块>/build && ctest --output-on-failure
# 模块: TaskFrame(32) StateMaster(6) Planning(2-3) Servo(2) Fieldbus(4) Launcher(1)
# 双 flavor: cmake .. -G Ninja -DBBT_ROBOT_FLAVOR=<flavor> 后重跑
```

8.2 相关文档

- docs/2026-10-13-system-state-recovery-delivery.md（重启恢复）
- docs/2026-10-13-auto-program-start-gate-delivery.md（启动门控）
- docs/2026-10-13-program-feeder-loop-delivery.md（供给器+循环）
- docs/2026-10-13-task-speed-override-integration-delivery.md（程序速度）
- docs/externalrequest/2026-10-13-dualarm-motion-stopping-wedge-rootcause.md